Projet Novapolis Résilience

Répartition des tâches pour un trinôme · guide de travail · ALMF51-BDML

En une phrase
Trois rôles qui travaillent en parallèle : **A** construit les fondations (données, graphe, scénarios, CLI), **B** s'occupe du réseau routier (parcours et plus courts chemins), **C** s'occupe du réseau de fibre, des circuits absorbants et des benchmarks. Tout le monde se met d'accord sur un **contrat d'interface** dès la première séance, puis chacun avance sans bloquer les autres.

1. Qui fait quoi (vue d'ensemble)

Rôle	Responsabilités principales	Fichiers	Barème
A · Fondations	Lecture et validation des CSV, classe Graphe, scénarios sur copie, générateur de jeux de données, CLI, README, assemblage du module 5	io_donnees.py, graphe.py, generateur.py, cli.py, indicateurs.py	Modélisation/archi (2) · Scénarios (3, partagé) · Reproductibilité
B · Routes	Module 1 (DFS, BFS, accessibilité, circuits) et module 2 (Dijkstra, Bellman–Ford, Floyd–Warshall, itinéraires, fermetures)	parcours.py, chemins.py	DFS/BFS (1) · Plus courts chemins (4)
C · Fibre & mesures	Module 4 (Union-Find, Kruskal, Prim, pannes), module 3 (circuits absorbants), benchmarks et graphiques	arbres.py, benchmarks.py, data/tests_negatifs/	Arbres/pannes (3) · Benchmarks (2)
Tous	Tests de son propre module, indicateurs du module 5, rapport, démo, relecture croisée	tests/, rapport	Tests (2) · Rapport/démo (3)

À savoir avant de commencer

- **A est sur le chemin critique** : B et C ont besoin de la classe Graphe et du petit jeu de données. A livre ces deux éléments en priorité absolue.
- En attendant, B et C codent leurs algorithmes sur des graphes construits à la main dans leurs tests.
- **Note individuelle** : le sujet prévient que la note baisse si un membre ne sait pas expliquer les algorithmes. Chacun doit pouvoir expliquer Dijkstra, Bellman–Ford et Kruskal, même s'il ne les a pas codés.

2. Séance de lancement (tous ensemble, environ 1 h)

Cette séance évite de tout réécrire au moment de l'intégration. Ne la sautez pas.

	Décision à prendre	Proposition
☐	**Langage**	Python 3 : heapq pour les tas, matplotlib pour les graphiques, pytest pour les tests, aucune bibliothèque de graphes (networkx interdit pour les algos, seulement pour vérifier si vous voulez).
☐	**Structure de données**	Listes d'adjacence : dict[id] → list[(voisin, poids)]. Mémoire O(|V|+|E|), cohérent avec les complexités du sujet. Matrice n×n seulement pour Floyd–Warshall.
☐	**Dépôt Git**	Un dépôt, une branche par personne, fusion au moins à chaque séance.
☐	**Format des CSV**	Celui du sujet (sites, routes, fibre, demandes) + un format pour scenarios.csv (voir A3). Statut : ouvert / ferme / travaux.
☐	**Contrat d'interface**	Signatures ci-dessous, à recopier dans graphe.py dès la séance.
☐	**Petit graphe de référence**	8 à 10 sommets, dessiné sur papier, avec les résultats attendus calculés à la main (distances, ACM, composantes). Il sert de test à tout le monde.

Arborescence du projet

```
novapolis/
├── data/  petit/  realiste/  genere/  tests_negatifs/
├── src/
│   ├── io_donnees.py    lecture, validation, écriture CSV            (A)
│   ├── graphe.py        classe Graphe + application des scénarios     (A)
│   ├── parcours.py      DFS, BFS, accessibilité, circuits             (B)
│   ├── chemins.py       Dijkstra, Bellman-Ford, Floyd-Warshall        (B)
│   ├── arbres.py        Union-Find, Kruskal, Prim, pannes             (C)
│   ├── indicateurs.py   indicateurs de résilience (module 5)          (A+B+C)
│   ├── benchmarks.py    mesures et graphiques                         (C)
│   ├── generateur.py    jeux synthétiques avec graine                 (A)
│   └── cli.py           interface en ligne de commande                (A)
├── tests/               un fichier de test par module
├── resultats/           CSV et PNG générés
└── README.md
```

Contrat d'interface (à ne plus modifier sans prévenir les autres)

```
class Graphe:
    def __init__(self, oriente: bool)
    def ajouter_sommet(self, id, **attributs)          # type, nom, coordonnées
    def ajouter_arc(self, u, v, poids, **attributs)    # en non orienté : ajoute dans les 2 sens
    def successeurs(self, u) -> list[tuple[id, float]]
    def sommets(self) -> list
    def arcs(self) -> list[tuple[id, id, float]]
    def copie(self) -> "Graphe"                        # copie profonde, l'original ne bouge jamais

# Tous les algorithmes de plus court chemin renvoient le même format :
#   (dist, pred)  avec  dist[v] = float('inf')   si v est inaccessible
#                       dist[v] = float('-inf')  si v est diminuable indéfiniment
def reconstruire_chemin(pred, source, cible) -> list | None
```

Personne A · Fondations

données, graphe, scénarios, CLI

Barème couvert : Modélisation, validation et architecture (2 pts) · part des scénarios (3 pts) et de la reproductibilité (2 pts)

	N°	Tâche	Détail et critère « c'est fini »
☐	A1	**Lecture des CSV**	io_donnees.py lit sites, routes, fibre, demandes, scenarios et sait réécrire un CSV. Fini quand le petit jeu se charge en une ligne.
☐	A2	**Validateur**	Rejette avec un message explicite : identifiant en double, arc vers un site inexistant (« routes.csv ligne 14 : site S99 inconnu »), colonne manquante, valeur non numérique, temps négatif dans un jeu métier. Fini quand chaque erreur a son test.
☐	A3	**Format de scenarios.csv**	Proposition : scenario,action,u,v,valeur avec action ∈ {fermer_route, modifier_temps, heure_pointe, fermer_fibre}. À documenter dans le README.
☐	A4	**Classe Graphe**	Respecte le contrat d'interface, orienté (routes) et non orienté (fibre). À livrer en premier : B et C en dépendent.
☐	A5	**Application des scénarios**	appliquer_scenario(graphe, scenario) travaille sur graphe.copie(). Test obligatoire : après application, le graphe de base est identique à avant.
☐	A6	**Petit jeu de données**	Le graphe de la séance de lancement, en CSV, avec ses résultats attendus écrits dans un fichier (sert de test à B et C).
☐	A7	**Jeu réaliste (≥ 100 sites)**	Ville synthétique : quartiers en grille, 3 ou 4 casernes, 2 hôpitaux, écoles, centres télécom, quelques sens uniques, temps normal et de pointe. Étiqueté SYNTHÉTIQUE dans le fichier et le README.
☐	A8	**Générateur paramétrable**	generer(n, densite, graine) pour ≥ 1000 sommets. Graine fixée et notée (random.seed). Même graine ⇒ même fichier, à tester.
☐	A9	**CLI**	argparse avec --donnees, --scenario, --operation (valider, accessibilite, itineraires, mst, panne, absorbant, resilience, bench). Fini quand toutes les commandes du README fonctionnent.
☐	A10	**Module 5 : orchestration**	Exécute les 3 scénarios (normal, fermeture d'un axe, panne fibre) et rassemble dans un tableau les indicateurs fournis par B et C.
☐	A11	**README**	Installation, commandes exactes copiables, description de chaque champ CSV, source et licence des données éventuelles.

Conseils pour A

- Python limite la récursion vers 1000 appels : évitez toute fonction récursive sur le gros graphe.
- Utilisez des identifiants en chaîne (S001, S002…) partout, jamais des indices qui changent selon le fichier.
- Pour le jeu réaliste, une grille avec quelques arcs supprimés et quelques sens uniques suffit. Si vous utilisez des données OpenStreetMap, citez la source et la licence ODbL.

Personne B · Routes

parcours et plus courts chemins

Barème couvert : DFS/BFS et accessibilité (1 pt) · Dijkstra, Bellman–Ford, Floyd–Warshall et chemins (4 pts)

Module 1 · Diagnostic d'accessibilité (parcours.py)

	N°	Tâche	Détail et critère « c'est fini »
☐	B1	**DFS itératif**	Version pile du cours (ch. 2, diapo 7). Pas de récursion.
☐	B2	**BFS + chemin min en arcs**	BFS avec prédécesseurs : donne le chemin qui minimise le nombre d'arcs.
☐	B3	**Accessibilité multi-casernes**	BFS multi-sources : toutes les casernes dans la file au départ. Renvoie les sites atteignables et la caserne d'origine.
☐	B4	**Circuits orientés**	DFS à 3 couleurs (blanc, gris, noir) : un arc vers un sommet gris = circuit. Renvoyer un circuit trouvé, pas seulement oui/non.
☐	B5	**Sites isolés / non atteignables**	Liste des sites sans arc, et des sites non atteints depuis les casernes.
☐	B6	**Bonus : composantes fortement connexes**	Kosaraju (2 DFS). Très utile pour le rapport, qui doit distinguer accessibilité, connexité et connexité forte.

Module 2 · Itinéraires des secours (chemins.py)

	N°	Tâche	Détail et critère « c'est fini »
☐	B7	**Dijkstra**	Tas binaire heapq, suppression paresseuse (on ignore les entrées périmées). Refuse un graphe avec un poids négatif (message clair).
☐	B8	**Bellman–Ford**	$\|V\|-1$ passes, arrêt anticipé si une passe ne change rien, passe supplémentaire de détection. Interface utilisée par C pour le module 3.
☐	B9	**Floyd–Warshall**	Matrice des distances + matrice des prédécesseurs P[i][j] (cours ch. 3, diapo 18).
☐	B10	**Reconstruction de chemin**	reconstruire_chemin commun aux 3 algos. Sortie : chemin, durée, nombre d'arcs, fermetures rencontrées.
☐	B11	**Itinéraire par demande**	Pour chaque ligne de demandes.csv : caserne retenue = la plus proche (Dijkstra depuis chaque caserne, ou multi-sources), puis le chemin.
☐	B12	**Scénario de fermeture**	Pour chaque demande : coût avant et après, retard absolu, retard relatif (après − avant) / avant, accessible ou inaccessible, chemin retenu.
☐	B13	**Comparaison des 3 algos**	Sur les mêmes instances, les distances doivent être identiques : en faire un test automatique.

Conseils pour B

- Le sujet demande d'expliquer quelle notion est utilisée pour chaque indicateur : « accessible depuis une caserne » n'est pas « connexe » et encore moins « fortement connexe » (cours ch. 1, diapos 25 à 31).
- Vérifiez à la main sur le petit graphe avant de passer au gros.
- Utilisez temps_pointe pour un scénario « heure de pointe » : c'est un scénario gratuit à ajouter.

Personne C · Fibre & mesures

arbres couvrants, absorbants, benchmarks

Barème couvert : Kruskal, Prim, Union-Find et pannes (3 pts) · Benchmarks et analyse critique (2 pts)

Module 4 · Conception du réseau de fibre (arbres.py)

	N°	Tâche	Détail et critère « c'est fini »
☐	C1	**Union-Find**	Compression de chemin + union par rang. Testé seul avant Kruskal.
☐	C2	**Kruskal**	Tri par (cout, u, v) pour départager les égalités de façon déterministe. Donne naturellement une forêt si le graphe n'est pas connexe.
☐	C3	**Prim**	Tas binaire. Relancé depuis chaque sommet non visité pour obtenir une forêt couvrante.
☐	C4	**Composantes de la fibre**	BFS sur graphe non orienté (ou Union-Find). Afficher les composantes non raccordées.
☐	C5	**Sortie lisible**	Tronçons choisis, coût total, nombre de composantes. Comparer Kruskal et Prim : même coût total, arêtes éventuellement différentes.
☐	C6	**Égalités de coûts**	Petit exemple où Kruskal et Prim donnent deux arbres différents de même coût. Explication : l'ACM n'est unique que si tous les poids sont distincts.
☐	C7	**Panne d'un tronçon**	Retirer une arête de l'ACM → 2 composantes (cours ch. 4, diapo 6). Les étiqueter par BFS, puis chercher dans le graphe INITIAL (sans l'arête en panne ni les interdites) l'arête la moins chère qui relie les deux côtés. Si aucune : le signaler.

Module 3 · Circuits absorbants (avec Bellman–Ford et Floyd–Warshall de B)

	N°	Tâche	Détail et critère « c'est fini »
☐	C8	**Graphe avec circuit accessible**	Circuit négatif atteignable depuis la source. Attendu : détecté.
☐	C9	**Graphe avec circuit non accessible**	Circuit négatif hors de portée de la source. Bellman–Ford depuis s ne le voit pas : le détecter avec une super-source virtuelle (reliée à tous les sommets, coût 0) ou la diagonale de FW (L[i][i] < 0).
☐	C10	**Distances $-\infty$**	Tout sommet atteignable depuis un sommet encore relâchable à la passe |V| reçoit $-\infty$ (propagation par BFS). Jamais de distance finie affichée dans ce cas.
☐	C11	**Présentation**	Préciser que ces coûts négatifs sont un jeu de test artificiel, pas des durées réelles.

Expériences (benchmarks.py)

	N°	Tâche	Détail et critère « c'est fini »
☐	C12	**Protocole de mesure**	time.perf_counter() autour de l'algorithme seul (chargement, affichage et génération exclus). 5 répétitions minimum, moyenne et écart-type.
☐	C13	**Plan d'expérience**	Tailles 100, 500, 1000, 2000, 5000 ; densités creuse ($m \approx 3n$) et dense. Mêmes graphes pour tous les algos. Graines notées.

	N°	Tâche	Détail et critère « c'est fini »
☐	C14	**Floyd–Warshall**	Seulement jusqu'à 200 à 400 sommets. Expliquer : $n^3 = 10^9$ opérations pour n = 1000, et n^2 cases en mémoire.
☐	C15	**Graphiques (≥ 3)**	Temps vs n (creux), temps vs n (dense), temps divisé par la complexité théorique (presque constant = théorie confirmée). Unités, légendes, échelle log-log.
☐	C16	**Analyse**	Comparer aux complexités du sujet et expliquer les écarts (constantes, cache, coût de Python).

3. Tâches partagées

	Tâche	Qui	Détail
☐	**Tests pytest**	Chacun	Chacun teste son module. Cas limites : graphe vide, sommet isolé, source inexistante, toutes les routes fermées, égalités de coûts, graphe non connexe.
☐	**Indicateurs routes**	B	Demandes devenues inaccessibles, retard moyen et maximal, proportion de sites sensibles encore accessibles.
☐	**Indicateurs fibre**	C	Nombre de composantes du réseau de fibre, coût de la réparation.
☐	**Tableau des 3 scénarios**	A	Normal / fermeture d'un axe important / panne fibre ou zone de travaux, avec au moins 2 indicateurs chacun.
☐	**Relecture croisée**	A→B, B→C, C→A	Chacun relit le code d'un autre et doit pouvoir l'expliquer.
☐	**Rapport (10–15 p.)**	Tous	Chacun écrit ses sections (voir plus bas), A assemble.
☐	**Démo (12 min)**	Tous	Environ 4 min chacun, chacun présente sa partie.
☐	**Relevé de contributions**	Chacun	Quelques lignes : modules codés, sections rédigées.

4. Planning conseillé

Phase	Qui	Objectif	Livrable de fin de phase
0 · Lancement	Tous	Contrat d'interface, Git, formats CSV, petit graphe sur papier	graphe.py squelette + petit graphe
1 · Développement	En parallèle	A : A1→A6 d'abord. B : B1→B10. C : C1→C7	Chaque algo passe ses tests sur le petit graphe
2 · Intégration	Tous	CLI branchée, jeux réaliste et généré, scénarios de bout en bout, module 3	Toutes les commandes du README marchent
3 · Résultats	C puis tous	Benchmarks, graphiques, tableau des scénarios, rédaction	Rapport complet
4 · Démo	Tous	Répétition chronométrée sur une machine propre	Démo de 12 min prête

5. Plan du rapport et responsables

Section	Contenu attendu	Qui
1. Contexte et modélisation	Deux graphes, pourquoi orienté et non orienté, unités	A
2. Données et architecture	Formats, validation, scénarios sur copie, structures de données choisies	A
3. Accessibilité	Résultats du module 1, notion utilisée pour chaque indicateur	B
4. Itinéraires des secours	Comparaison des 3 algos, effet des fermetures	B
5. Circuits absorbants	Les 2 graphes de test, détection, distances $-\infty$	C
6. Réseau de fibre	ACM, égalités, pannes et raccordement	C
7. Résilience	Tableau des 3 scénarios et interprétation	Tous
8. Benchmarks	Protocole, graphiques, comparaison à la théorie	C
9. Limites et recommandations	Données synthétiques, temps fixes, pas de capacité ni de disponibilité des véhicules, priorités ignorées... et recommandations pour la ville	Tous

Checklist finale avant de rendre

- Un résultat qui ne peut pas être régénéré à partir des fichiers et commandes fournis ne sera pas validé.
- Les perturbations s'appliquent toujours sur une copie : le scénario de référence ne change jamais.
- Chaque membre doit pouvoir expliquer tous les algorithmes et les choix de conception.
- Le sujet insiste : il ne suffit pas d'obtenir un chemin ou un arbre, il faut le justifier, mesurer l'effet des perturbations et discuter les limites du modèle.